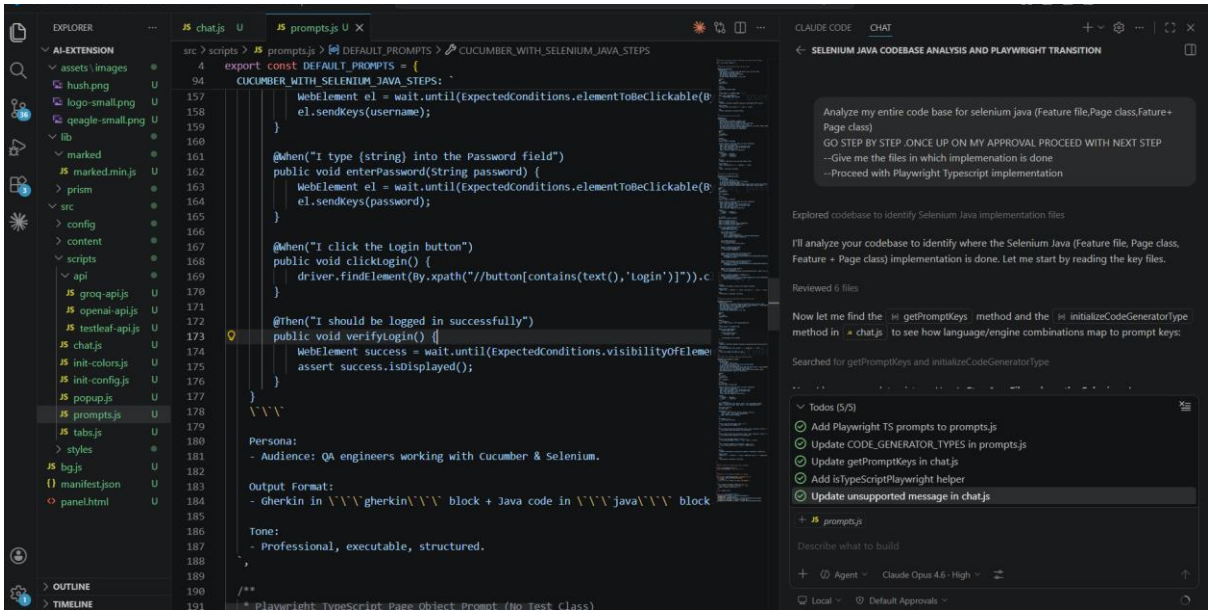
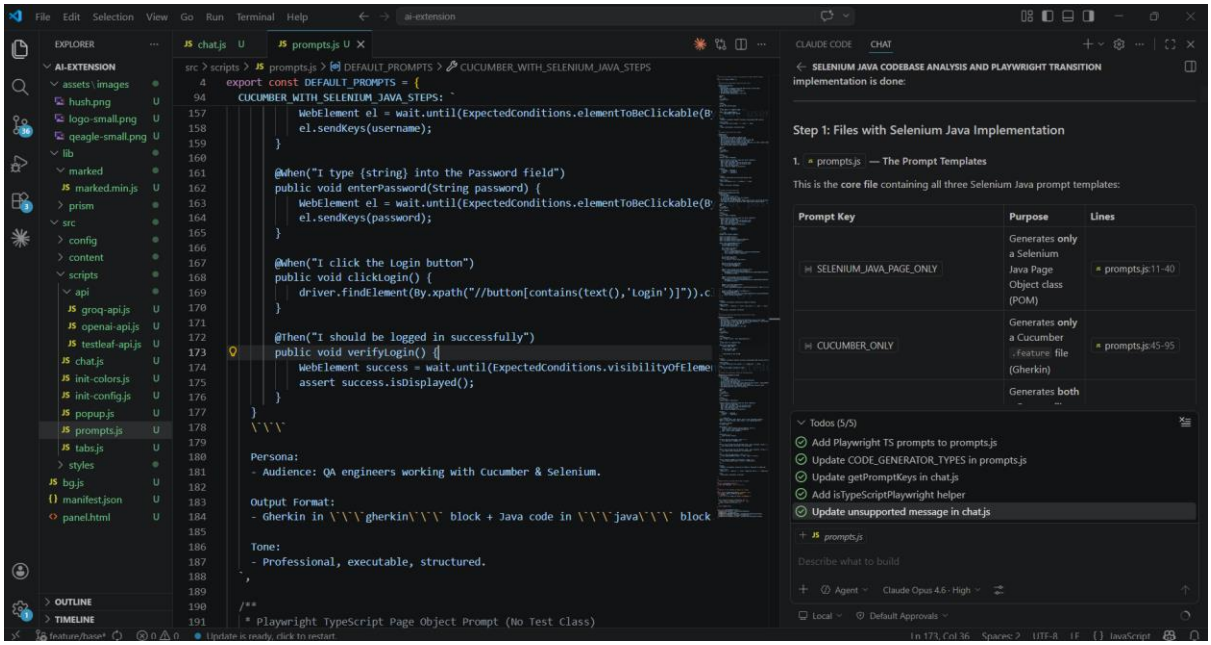
Analyze my entire code base for selenium java (Feature file,Page class,Fature+ Page class)

GO STEP BY STEP .ONCE UP ON MY APPROVAL PROCEED WITH NEXT STEP

--Give me the files in which implemenation is done

--Proceed with Playwright Typescript implementation

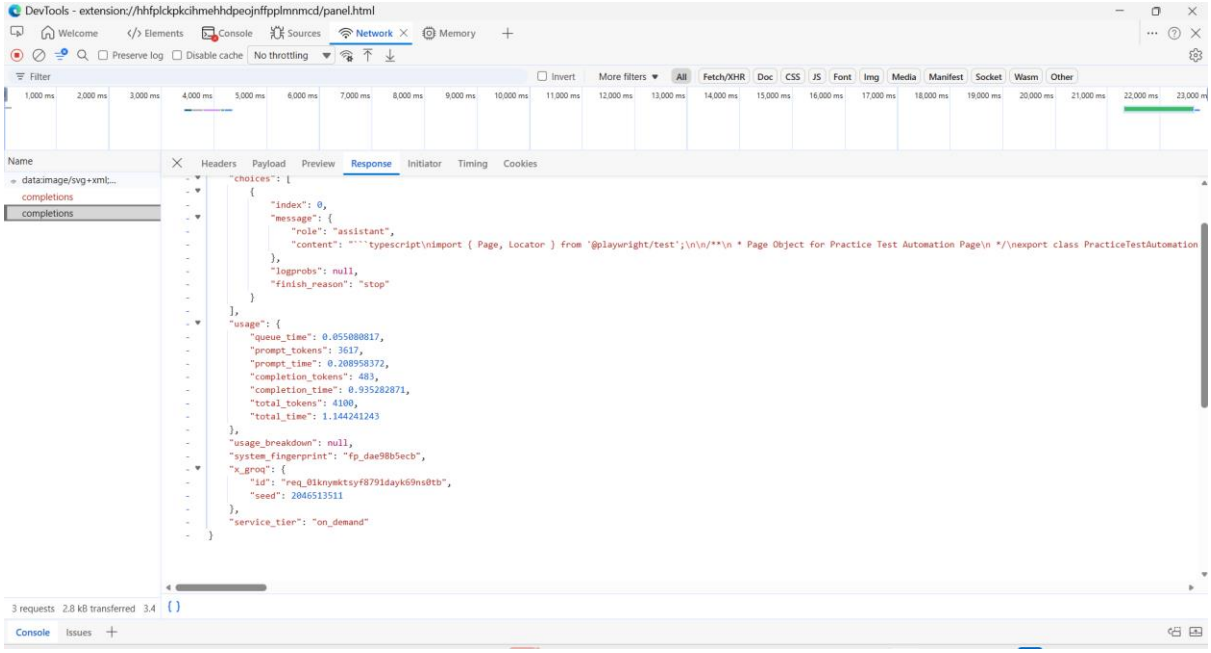




```
export const DEFAULT_PROMPTS = {
  /*
  CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS: `
    Instructions:
    - Generate BOTH:
      1. A Cucumber .feature file.
      2. A TypeScript step definition file using Playwright.
    - Do NOT include Page Object code.
    - Step defs must use Playwright's async API with page.locator(), page.getByR
    - Use async/await for all Playwright interactions.
    - Use Scenario Outline with Examples table (South India realistic data).
    - Use @playwright/test expect for assertions.

    Context:
    DOM:
    \`\`\`html
    \${domContent}
    \`\`\`
    URL: \${pageUrl}

    Example:
    \`\`\`gherkin
    Feature: Login to Application

    Scenario Outline: Successful login with valid credentials
      Given I open the login page
      When I type "<username>" into the Username field
      And I type "<password>" into the Password field
      And I click the Login button
      Then I should be logged in successfully

    Examples:
      | username   | password  |
      | "admin"    | "admin123"|
    \`\`\`

    \`\`\`typescript
```

```
PLAYWRIGHT_TS_PAGE_ONLY: `
    Instructions:
    - Generate ONLY a Playwright TypeScript Page Object Class (n
    - Use Playwright's Locator API (page.locator(), page.getByRo
    - Use async/await for all Playwright interactions.
    - Add JSDoc comments for methods & class.
    - Use meaningful method names.
    - Do NOT include explanations or test code.
```

```
"error": {
    "message": "The model `deepseek-r1-distill-llama-70b` has been decommissioned and is no longer supported. Please refer to https://console.groq.com/docs/deprecations for a recommendati
    "type": "invalid_request_error",
    "code": "model_decommissioned"
}
}
```



Generated code :

```
import { Page, Locator } from '@playwright/test';


/**
 * Page Object for Practice Test Automation Page
 */
export class PracticeTestAutomationPage {
  readonly page: Page;


  constructor(page: Page) {
    this.page = page;
  }


  /**
   * Locator for the post title
   */
  get postTitle(): Locator {
```

```
    return this.page.locator('.post-title');
}

/**
 * Locator for the image in the post
 */
get postImage(): Locator {
    return this.page.locator('.wp-block-image img');
}

/**
 * Locator for the paragraphs in the post
 */
get postParagraphs(): Locator {
    return this.page.locator('p');
}

/**
 * Locator for the links in the post
 */
get postLinks(): Locator {
    return this.page.locator('a');
}

/**
 * Locator for the mailpoet form
 */
get mailpoetForm(): Locator {
```

```
    return this.page.locator('#mp_form_below_posts7');
}

/**
 * Locator for the name input field in the mailpoet form
 */
get nameInput(): Locator {
    return this.page.locator('#form_first_name_7');
}

/**
 * Locator for the email input field in the mailpoet form
 */
get emailInput(): Locator {
    return this.page.locator('#form_email_7');
}

/**
 * Locator for the submit button in the mailpoet form
 */
get submitButton(): Locator {
    return this.page.locator('.mailpoet_submit');
}

/**
 * Fill the name input field in the mailpoet form
 * @param name The name to fill in
 */
```

```
  async fillName(name: string): Promise<void> {
    await this.nameInput.fill(name);
  }

  /**
   * Fill the email input field in the mailpoet form
   * @param email The email to fill in
   */
  async fillEmail(email: string): Promise<void> {
    await this.emailInput.fill(email);
  }

  /**
   * Click the submit button in the mailpoet form
   */
  async submitForm(): Promise<void> {
    await this.submitButton.click();
  }
}
```

<u>Featurefile+ Page class:</u>

```
Feature: Verify Post Title

 Scenario Outline: Verify post title with different titles
  Given I open the practicetestautomation website
  When I verify the post title as "<title>"
  Then I should see the post title as "<title>"
```

```
  Examples:
    | title     |
    | "Hello"    |
    | "Welcome"  |
    | "Namaste"  |

import { Given, When, Then, Before, After } from '@cucumber/cucumber';
import { chromium, Browser, Page, expect } from '@playwright/test';

let browser: Browser;
let page: Page;

Before(async () => {
   browser = await chromium.launch({ headless: false });
   const context = await browser.newContext();
   page = await context.newPage();
});

After(async () => {
   await browser?.close();
});

Given('I open the practicetestautomation website', async () => {
   await page.goto('https://practicetestautomation.com/');
});

When('I verify the post title as {string}', async (title: string) => {
   // No action required for this step
```

```
});

Then('I should see the post title as {string}', async (title: string) => {
  await expect(page.locator('h1.post-title')).toContainText(title);
});
```